Daffodil International University

Department of Computer Science and Engineering (CSE)

CSE422: Computer Graphics Lab

Spring 26

PROJECT SKETCH

Project Title: Forza Horizon 2099

Role / Module	Name	Student ID
Background Scene	Safayet Hossain Nahin	0242220005101569
Player Car	Shoeb Akter Nafiz	0242220005101898
Enemy Cars	Sumaiya Rahman Moon	0242220005101589
HUD & UI	Ridita Sultana	0242220005101594
Collision & Logic	Refat Pasha	0242220005101568

Submission Date: March 18, 2026

1. Introduction:

"Forza Horizon 2099" is an interactive car racing game developed using C++ and OpenGL (FreeGLUT). The game presents a top-down, three-lane highway where the player controls a red sports car and must avoid oncoming traffic to survive as long as possible. The score increases continuously while the game is running, and the difficulty scales through five levels as the enemy cars progressively increase in speed.

The project is designed to directly apply all the core graphics concepts covered throughout the course. It uses output primitives (points, lines, polygons) to construct every visible object, implements three required graphics algorithms (DDA, Bresenham, Midpoint Circle), applies all major 2D geometric transformations (translation, rotation, scaling), and incorporates real-time animation through GLUT's timer mechanism. The interactive game was chosen specifically because it demands all of these techniques simultaneously in a real-world engaging context.

The core scene rendering - background, player car, enemy cars, and all three required algorithms are fully functional. The remaining work covers the game-over flow, HUD polish, collision response animation, and level progression, all of which are actively in progress.

2. Motivation:

Our team selected the Interactive Car Racing Game theme for the following reasons:

i. Games provide a natural, real-world context where all required course topics graphics primitives, line algorithms, circle algorithms, 2D transformations, and animations are used simultaneously and meaningfully rather than in isolation.
ii. A lane-based car avoidance game creates an obvious need for all three required algorithms. Lane divider lines drawn with DDA, enemy car outlines drawn with Bresenham, and circular wheels drawn with the Midpoint Circle algorithm each serve a clear visual and functional purpose in the scene.
iii. The interactive nature of the game directly addresses the course requirement for meaningful real-world application, moving beyond a static drawing into a system that responds to user input and changes state over time.
iv. We were also motivated by the challenge of building something visually and technically impressive within the constraints of what we have learned so far.

3. Project Theme:

"Forza Horizon 2099" is a top-down, 2D, real-time car avoidance game rendered entirely with OpenGL primitives. The game window (700 × 800 pixels) shows a three-lane highway from a bird's-eye perspective. The player's red car is positioned near the bottom of the screen and can move left or right between the three lanes using the keyboard. Three enemy cars one blue, one green, one yellow appears at the top of the screen and drive downward at increasing speeds. The player must dodge them. Each collision costs one life. The player starts with three lives. When all lives are lost, the game ends and the final score is displayed.

The road itself scrolls using animated DDA-drawn lane markings, creating the illusion that the player's car is driving forward even though its Y position is fixed. The environment includes a gradient sky, moving clouds, and decorative trees on both sides of the road to give depth and context to the scene.

Game progression is handled through five levels. Every 300-score point, the level increases, which raises the base speed of all enemy cars. The score increments every frame while the game is running, so surviving longer always yields a higher score.

Key game states include:

i. A Start Screen.
ii. The main Play state.
iii. A Pause state.
iv. A Game Over screen.

All navigable through keyboard inputs (e.g., SPACE, R, ESC).

The complete project, when finished, will demonstrate: three graphics algorithms (DDA, Bresenham, Midpoint Circle), four types of 2D transformations (translation, rotation, scaling, and a visual tilt using rotation on lane change), continuous animation (scrolling road, moving clouds, downward enemy movement, explosion particles), and user interaction through keyboard input.

4. Sketch of Project Scene:

The game window is divided vertically into three visual zones:

i. The sky zone at the top.
ii. The active road in the center.
iii. The HUD overlay at the very top edge.

The illustration below represents a rough layout of the game.

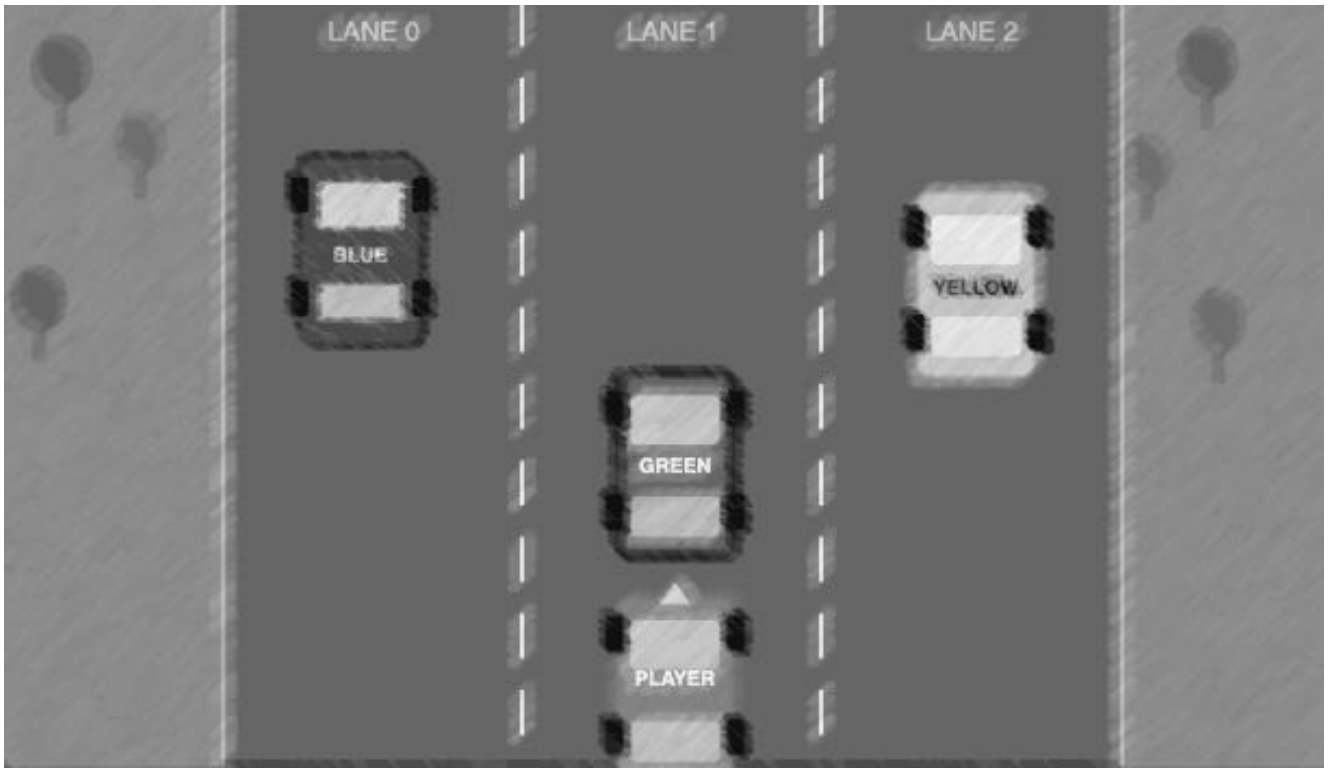

Fig: Project sketch illustration

Component overview:

i. Background (Sky zone): Gradient blue background, 4 translating white clouds, 6 static trees (3 per side).
ii. Background (Road zone): Dark grey asphalt, two white edge lines, two DDA-drawn yellow dashed lane dividers.

iii. Enemy cars: Three colored cars (blue, green, yellow), one per lane, moving downward.
iv. Player car: Red car with Midpoint Circle wheels, controlled by arrow keys, positioned near the bottom.
v. HUD overlay: Semi-transparent panels at the top showing score, level, and lives (heart icons).

5. Detailed Description of Project Scene Components:

i. **Background:**

The sky is drawn as a single GL_QUADS polygon with two different glColor3f calls, creating a gradient from light blue near the horizon (y = -5) to deep blue at the top (y = 40). Two additional green quads represent the grassy shoulders on either side of the road. The road itself is a dark grey rectangle from x = -18 to x = +18 spanning the full screen height, bordered by two white GL_LINES edge markers.

The lane markings are yellow dashed lines drawn using the DDA Line Algorithm. A float variable dashOffset accumulates by 0.40 units per frame and wraps when it reaches the dash period (9.0 units), which makes the dashes appear to scroll continuously downward simulating forward movement.

Four cloud objects each store an x, y, scale, and speed. Every frame, each cloud's x position increases by its speed value (translation). When x exceeds +42, it wraps to -42 so it re-enters from the left. Clouds are drawn as four overlapping GL_POLYGON circles at slightly different offsets to form a puff shape. Six decorative trees (GL_TRIANGLES for foliage, GL_QUADS for trunks) are placed statically at fixed positions outside the road boundaries.

ii. **Player Car:**

The player car is a detailed polygon-based model drawn entirely from GL_QUADS and GL_POLYGON shapes. It consists of: a main body, a darker cabin/roof, a blue windshield, a slightly lighter rear window, two circular headlights with glow circles, two red tail light panels, and a dark side stripe. The car sits at a fixed y position (y = −28) and can only move horizontally between three preset lane positions (laneX[0] = −12, laneX[1] = 0, laneX[2] = +12).

Lane movement is handled through smooth interpolation. When the player presses left or right, the targetX is set to the new lane's X coordinate. Each frame, playerX moves 18% of the remaining distance toward targetX, giving a smooth slide effect rather than an instant jump. A rotation tilt of ±6 degrees is applied using glRotatef when a lane change begins, and it decays by 15% per frame back to zero.

Each of the four wheels is drawn in layers: a black outer tyre using filledCircle (GL_POLYGON), a Midpoint Circle Algorithm outline plotted as GL_POINTS on the tyre edge, a silver rim, a white hub, and five GL_LINES spokes radiating outward.

iii. **Enemy Cars:**

Three enemy cars are maintained in an array. Each store: x position, y position, current speed, lane index, RGB color, and an active flag. The three starting colors are blue (0.15, 0.35, 0.90), green (0.10, 0.72, 0.20), and yellow (0.95, 0.75, 0.10). Each enemy car is drawn using the same polygon structure as the player car but with a different color scheme and without the same level of detail.

A key feature of each enemy car's drawing is that its four edges - top, bottom, left, and right - are drawn using the Bresenham Line Algorithm (via a floating-point wrapper that scales world coordinates to an integer grid). This gives enemy cars a visually distinct outlined appearance compared to the player's smooth polygon body.

Every frame, each enemy car's y position decreases by its own speed plus a level-based base speed (0.28 + level × 0.07). When an enemy car's y drops below −50 (off the bottom of the screen), it is respawned

at the top (y = 50 + random 0..19) in a randomly chosen lane with a new randomized speed between 0.18 and 0.40.

iv. **HUD & UI Screens:**

The HUD consists of three semi-transparent panels rendered at the top of the screen. The score panel (top-left) uses a drawPanel function that applies glTranslatef and glScalef to position and size a unit square - demonstrating 2D transformation in the HUD layer. The level panel (top-center) shows the current level number. The lives panel (top-right) renders the text 'LIVES:' followed by heart-shaped icons, one per remaining life. Each heart is drawn using two GL_POLYGON semicircles and a GL_TRIANGLES bottom point, again positioned and sized via glTranslatef and glScalef.

The Pause Overlay draws a semi-transparent black quad over the entire screen followed by a panel with 'PAUSED' and 'Press SPACE to Resume' text. The Start Screen is complete and includes a decorative road strip, title panel, course information, and a full control listing. The Game Over screen design is drafted - it shows the final score and restart instructions - but the visual polish and transition timing are still being refined.

v. **Collision Detection & Game Logic:**

Collision detection uses AABB (Axis-Aligned Bounding Box) testing. Every frame, the absolute horizontal and vertical distances between the player's center and each enemy car's center are compared against the sum of their respective half-widths and half-heights. If both axes overlap, a collision is registered. The player's bounding box is set to half-width 2.0, half-height 3.2 - slightly smaller than the drawn car to make hits feel fair.

On collision, the player loses one life, an explosion particle system is triggered, the offending enemy car is pushed off screen, and an invincibility timer is set to 100 frames to prevent repeated hits from a single incident. The explosion system spawns 22 particles, each with a random direction vector, speed, size, and rotation speed. Each frame, every live particle is translated to its current position, rotated by its rotation angle using glRotatef, and scaled down proportionally to its remaining life using glScalef - all three 2D transformations applied per particle per frame.

The game state machine (STATE_START → STATE_PLAY → STATE_PAUSE → STATE_OVER) is implemented but the transition from STATE_OVER back to a clean game restart still has a bug where enemy positions are not always correctly reset. This is currently being debugged.

6. Transformation Plan:

The following 2D geometric transformations are applied across the project. All transformations use glPushMatrix / glPopMatrix to isolate their effects so that one object's transformation does not affect any other object drawn afterward.

Transformation	Applied To	How It Works in Code
Translation	Player car, enemy cars, clouds, HUD panels, explosion particles	glTranslatef(x, y, 0) moves each object to its current world coordinate position every frame. For the player car, playerX is updated each frame toward targetX. For enemies, their stored y value decreases each frame. For clouds, x increases each frame and wraps.
Rotation	Player car on lane change, explosion particles	glRotatef(playerTilt, 0, 0, 1) rotates the player car around the Z-axis when a lane change is initiated. playerTilt is set to ±6 degrees and decays back to 0 each frame. For explosion

		particles, particle.angle increases each frame, creating a spinning tumble effect.
Scaling	HUD panels, explosion particles	glScalef(bw, bh, 1) in drawPanel() scales a unit 1×1 square to the exact width and height of each HUD panel. For explosion particles, glScalef(sz*life, sz*life, 1) shrinks each particle as its life value fades from 1.0 to 0.0, creating a natural fade-out.
Reflection (conceptual)	Road edge symmetry	The left and right road edges and tree positions are mirror images of each other, drawn symmetrically around x = 0. This demonstrates the principle of reflection even though it is implemented through symmetric coordinate values rather than a glScale(-1,1,1) call.

7. Project Animation Plan:

All animation in the project is driven by a single GLUT timer callback, glutTimerFunc(16, timerFunc, 0), which fires approximately every 16 milliseconds (targeting 60 frames per second). Each call to timerFunc updates all animated values and then calls glutPostRedisplay() to trigger a redraw. The following animation techniques are used:

i. Road Scroll Animation: The variable dashOffset increases by 0.40 units every frame. The DDA-drawn lane dashes are repositioned each frame relative to this offset. When dashOffset reaches the dash period (9.0 units), it wraps back to 0. This creates a continuous scrolling effect that convincingly simulates the player's car driving forward, even though the car's y coordinate never changes.

ii. Cloud Translation: Each of the four cloud objects has an individual speed value (ranging from 0.015 to 0.030 units per frame). Each frame, cloud.x += cloud.speed. When cloud.x exceeds +42, it resets to -42 so clouds appear to drift continuously from left to right across the sky.

iii. Enemy Car Downward Movement: Each enemy car's y coordinate decreases by (enemy.speed + baseSpeed) every frame, where baseSpeed = 0.28 + level × 0.07. This makes all enemies move faster as the game level increases. When an enemy exits the bottom of the screen ($y < -50$), it is respawned at the top with a new random lane and speed.

iv. Player Car Smooth Lane Transition: When the player presses a direction key, targetX is set to the new lane's x coordinate. Each frame, playerX += (targetX − playerX) × 0.18. This exponential interpolation (lerp) makes the car smoothly slide to the target lane rather than jumping instantly. The tilt angle is also animated - it is set to ±6 degrees at the start of a move and decays by 15% per frame (playerTilt *= 0.85).

v. Explosion Particle Animation (In Progress): On collision, 22 particles are spawned with randomized directions and speeds. Each frame per live particle: position is updated by velocity, vertical velocity is decreased by 0.012 (gravity simulation), life decreases by 0.032, and rotation angle increases by the particle's rotSpd. The particle is then drawn as a small square with glTranslatef, glRotatef, and glScalef applied so it appears to fly outward, spin, and shrink simultaneously as it fades. Currently being polished to ensure smooth fade-out timing.

All animation is contained within the timerFunc callback and the individual update functions it calls (updateRoadMarkings, updateClouds, updateEnemyCars, checkCollisions). The display function only reads the current state and renders it - it does not update any values itself. This clean separation of update and draw logic is intentional.

8. Program Structure Plan:

i. Architecture:

The entire project is contained in a single main.cpp file, structured into five clearly labelled sections corresponding to the five team members, followed by the GLUT setup and callbacks. Each section is preceded by a large block comment identifying the member and their responsibility. Functions defined in later sections can call functions defined in earlier sections because C reads top to bottom - this dependency order was planned deliberately.

Code Section	Responsibility
Global variables & constants	Game state, lane positions, #defines for states
Background	Sky, road, DDA lane markings, clouds, trees
Player Car	filledCircle, midpointCircleOutline, drawWheel, drawPlayerCar, movement
Enemy Cars	bresenhamLine, bresenhamF, drawEnemyCar, initEnemyCars, updateEnemyCars
HUD & Screens	renderText, drawPanel, drawHeart, drawScore, drawLives, all screens
Game Logic	spawnExplosion, drawExplosion, aabbCollide, checkCollisions, initGame
GLUT Main & Callbacks	display, timerFunc, keyboard, specialKeys, reshape, main

ii. Algorithms Used and Their Scenarios:

- **DDA (Digital Differential Analyzer)**

Applied in drawRoadMarkings(). Given the start and end y coordinates of each lane dash, the algorithm computes dx and dy, determines the number of steps as max(|dx|, |dy|), then increments x and y by dx/steps and dy/steps each step. Since the lane dashes are perfectly vertical, dx = 0 and dy = dashLength, making the increment xi = 0, yi = 1.0. GL_POINTS are plotted at each step. The scroll is achieved by offsetting all starting y positions by the animated dashOffset variable.

- **Bresenham Line Algorithm**

Applied in drawEnemyCar() via bresenhamF(). The four edges of each enemy car body (top, bottom, left, right) are drawn using Bresenham's integer error-term method. The algorithm tracks an error term (err = dx − dy) and uses it to decide at each step whether to advance in the primary or secondary direction. The floating-point world coordinates are converted to integers by multiplying by a scale factor of 80 before passing to the algorithm, and GL_POINTS are plotted at each integer position.

- **Midpoint Circle Algorithm**

Applied in midpointCircleOutline(), which is called from drawWheel() to draw the tyre outline. The algorithm starts at (0, r) with decision parameter d = 1 − r, and each iteration plots 8 symmetric points (one per octant of the circle) using reflections across both axes and the diagonals. The decision parameter d determines whether y decreases or not: if d < 0, d += 2xi + 3; otherwise, d += 2(xi − yi) + 5 and yi decreases. Points are plotted as GL_POINTS against the tyre's filled polygon circle.

iii. Completed vs. Planned (Progress at 60%):

Feature / Component	Status	Notes
Sky gradient and grassy road shoulders	**Done**	Renders correctly at all window sizes
DDA Algorithm - scrolling road lane dashes	**Done**	Dash offset wraps cleanly each frame
Clouds (translation animation, wrap-around)	**Done**	All 4 clouds moving at different speeds
Side trees (static decorative triangles)	**Done**	3 trees left, 3 right, outside road
Road rectangle and white edge lines	**Done**	Correct dimensions and color
Player car body (polygons, multiple parts)	**Done**	Body, roof, windows, lights all drawn
Midpoint Circle - wheel tyre outlines	**Done**	8-point symmetry plots correctly on all 4 wheels
Smooth lane change (lerp + tilt rotation)	**Done**	Tilt decay and lerp both working
Enemy car drawing (3 cars, 3 colors)	**Done**	All three lane positions rendering
Bresenham Algorithm - enemy car outlines	**Done**	Scale factor correctly converts world to integer
Enemy downward movement and level speed scaling	**Done**	Base speed formula tested at levels 1-5
Enemy respawn (random lane, random speed)	**Done**	Respawn position and timing verified
HUD score panel (score counter, formatted text)	**In Progress**	Counter works, panel transparency needs tuning
HUD lives panel (heart icons per life)	**In Progress**	Hearts render, spacing at edge cases needs fix
HUD level indicator	**In Progress**	Displays correctly, font size being adjusted
Start screen layout and controls display	**In Progress**	Text and road strip done, styling ongoing
AABB collision detection logic	**In Progress**	Overlap check triggers, bounding box tuning needed
Invincibility timer after hit	**In Progress**	Timer counts down, edge case at level 5 needs fix
Explosion particle system (translate/rotate/scale)	**In Progress**	Particles spawn and move; fade timing being refined
Game Over screen	**Planned**	Layout designed, not yet implemented in code
STATE_OVER to STATE_PLAY restart (initGame bug)	**Planned**	Enemy reset issue identified, fix in progress
Level progression display feedback to player	**Planned**	Flash effect planned on level-up, not started

9. Progress Status:

As of March 18, 2026 we have completed approximately 60% of the total project. The core structural foundation - scene rendering, all three required algorithms, player and enemy car drawing, and smooth player movement - is fully operational. The second 40% consists of the collision response system, HUD

finalization, the game-over screen, and restart flow, all of which are at various stages of active development.

Completed Portions:

i. **Nahin (Background):** The full background scene is done. Sky gradient, scrolling DDA lane markings, animated clouds, road with edge lines, and static trees all render correctly. The dashOffset wrapping and cloud x-wrap have been tested and work without visual glitches.
ii. **Nafiz (Player Car):** The player car draws with all polygon layers. The Midpoint Circle wheel outlines plot correctly. Keyboard input for lane change works with both arrow keys and A/D. Smooth lerp and tilt animation are functional.
iii. **Moon (Enemy Cars):** All three enemy cars draw with Bresenham outlines. Downward movement and respawn logic work correctly. Level-based speed scaling has been verified at levels 1 through 5.
iv. **Ridita (HUD):** Score counter increments and displays correctly. Lives hearts render in the correct positions. Start screen is visually complete. Level indicator displays. Minor panel transparency and text alignment issues remain.
v. **Refat (Collision):** AABB collision detection triggers correctly in test cases. The invincibility timer prevents rapid re-hits. Particle spawning works and particles move outward with gravity. Fade and rotation animations are partially done.

In-Progress Portions:

i. The explosion particle fade-out (scaling with life) is partially working but the timing looks abrupt - we are adjusting the decay constant.
ii. The game restart flow (initGame called on 'R' key press) correctly resets score, lives, and level, but enemy y positions are not always reset cleanly, causing two enemies to sometimes start stacked. Being actively debugged.
iii. HUD panel alpha blending looks slightly different on different Mac screen settings. We are adjusting the glColor4f alpha values.

Evidence of Progress:

Screenshots of the current build are described below:

i. Screenshot 1: Start Screen - title panel, road strip, and controls list visible.

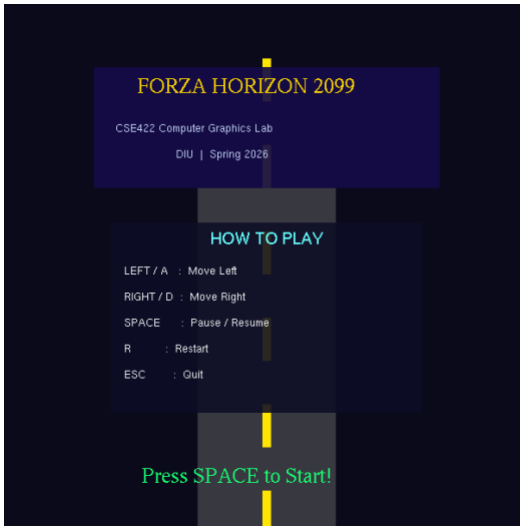

Fig: Start Screen

ii. Screenshot 2: Gameplay - three enemy cars, player car, scrolling lane markings, and HUD panels.

Fig: Gameplay

iii. Screenshot 3: Lane change - player car with visible tilt rotation applied.

Fig: Lane Change

iv. Screenshot 4: Explosion - particle system visible on screen after a test collision.

Fig: Explosion and Ending

GitHub Repository: https://github.com/beyounafiz/forza-horizon-2099

10. Expected Challenges:

The following challenges have been identified for the remaining 40% of the project, along with our planned approaches to overcome them:

i. **Challenge 1: Explosion Particle Fade Timing**

Currently the particles disappear too abruptly at the end of their life cycle. The decay constant (life -= 0.032 per frame) means the particle lives for about 31 frames at 60fps, but the scale fade (glScalef with sz * life) collapses to nothing before the color fade catches up. We plan to separate the scale fade and color fade curves so they end at the same visual moment.

ii. **Challenge 2: Clean Game Restart**

The initGame() function calls initPlayerCar(), initEnemyCars(), and initBackground() to reset all modules. The enemy reset sometimes leaves two cars at the same y position because rand() % 3 can return the same lane twice. We will add a check to ensure each enemy starts in a distinct lane on restart.

iii. **Challenge 3: Alpha Blending Consistency**

The semi-transparent HUD panels use glEnable(GL_BLEND) with GL_SRC_ALPHA and GL_ONE_MINUS_SRC_ALPHA. On some Mac display configurations, the transparency renders differently than expected. We will test with different alpha values and potentially switch to drawing the panels before opaque objects to fix z-order issues.

iv. **Challenge 4: High-Speed Collision Miss**

At level 5, enemy base speed reaches 0.63 units per frame. Given the player bounding box half-height of 3.2, an enemy technically could slip through the player's position in a single frame if the speed is high enough. We plan to clamp the maximum enemy speed or add a simple swept-AABB check (testing the rectangle swept over one frame) to prevent tunnelling at high speeds.

v. **Challenge 5: Integration of All Modules**

Since each member has been developing their section somewhat independently, the final integration session may surface issues where variable names clash or function call order causes bugs. We have scheduled a joint coding session before the deadline to merge all parts and run a full test of every game state.

11. Conclusion

"Forza Horizon 2099" is an interactive OpenGL car racing game that brings together all the three mandatory graphics algorithms (DDA for lane markings, Bresenham for enemy car outlines, and Midpoint Circle for wheel drawing) are each applied where they make the most visual and functional sense, rather than being added as isolated demonstrations. The four 2D transformations (translation, rotation, and scaling for all moving objects; conceptual reflection in the symmetric road layout) are used continuously throughout the game loop, not just in a single place.

As of the project sketch submission, 60% of the project is complete. The full background scene, both car models, all three algorithm implementations, smooth player movement, and enemy movement with level scaling are all working. The remaining work - collision response polish, HUD finalization, the game-over screen, and the restart flow - is clearly defined, distributed fairly among team members, and expected to be finished well before the final project demonstration.

The modular structure of the codebase (five clearly labelled sections, each owned by one team member) ensures that progress on each part can proceed independently and that the contribution of each member is clearly identifiable. We are confident that the final product will be a complete, working, and visually polished game that fully meets the requirements of the project guidelines.